A Tutorial on Graph-bashed SLAM

Yubao Liu

Grisetti, G., Kummerle, R., Stachniss, C., & Burgard, W. (2010). **A tutorial on graph-based SLAM.** *IEEE Intelligent Transportation Systems Magazine*, *2*(4), 31–43. https://doi.org/10.1109/MITS.2010.939925

G2O: Graph Optimization

[1] Grisetti, G., Kummerle, R., Stachniss, C., & Burgard, W. (2010). **A tutorial on graph-based SLAM**. *IEEE Intelligent Transportation Systems Magazine*, *2*(4), 31–43.

[2] Kümmerle, R., Grisetti, G., Strasdat, H., Konolige, K., & Burgard, W. (2011)**. G2o: A general framework for graph optimization**. In *Proceedings - IEEE International Conference on Robotics and Automation* (pp. 3607–3613). IEEE.

[3] Grisetti, G., Kümmerle, R., Strasdat, H., & Konolige, K. (2012). **g2o: A general framework for (Hyper) graph optimization**. https://github.com/RainerKuemmerle/g2o.git

Fig. 9. Intel Research Lab. Left: Unoptimized pose graph overlayed on top of the resulting map. Right: The optimized pose graph and the resulting consistent map.

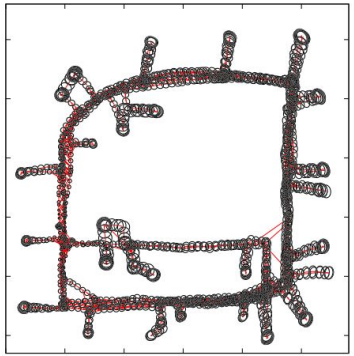


Fig. 10. Pose uncertainty estimate for a real-world data set.

Abstract

- simultaneous localization and mapping (SLAM) problem

Being able to build a **map** of the environment and to **simultaneously localize within this map** is an essential skill for mobile robots **navigation in unknown environments** in absence of external referencing systems such as GPS.

One intuitive way of formulating SLAM is to use a **graph** whose **nodes** correspond to the poses of the robot at different points in time and whose **edges** represent constraints between the poses.

The latter are obtained from **observations** of the environment or from **movement** actions carried out by the robot. Once such a graph is constructed, the map can be computed by finding the spatial configuration of the nodes that is mostly consistent with the measurements modeled by the edges.

Contribution

- Provide an introductory description to the graph-based SLAM problem.
- Discuss a state-of-the-art solution that is based on least-squares error minimization and exploits the structure of the SLAM problems during optimization.
- The goal of this tutorial is to enable the reader to implement the proposed methods from scratch

PROBABILISTIC FORMULATION OF SLAM

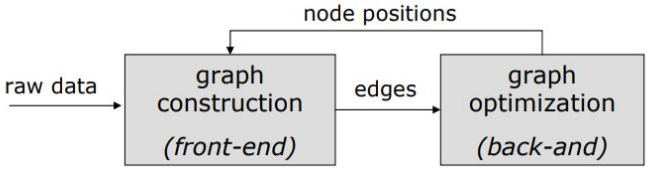
Solving the SLAM problem consists of **estimating the robot trajectory** and t**he map of the environment as the robot moves in it**.

Due to the **inherent noise in the sensor measurements**, a SLAM problem is usually described by means of **probabilistic tools**.

The robot is assumed to move in an **unknown environment**, along a **trajectory** described by the sequence of random variables $x_{1:T} = \{x_1, \ldots, x_T\}$.

While **moving**, it acquires a sequence of **odometry measurements** $u_{1:T} = \{u_1, \ldots, u_T\}$ and **perceptions** of the environment $z_{1:T} = \{z_1, \ldots, z_T\}$.

Full SLAM Problem



- Full SLAM problem
  - Estimating the posterior probability of the robot's trajectory x1:T
  - Estimating the map m of the environment given all the measurements plus an initial position x0

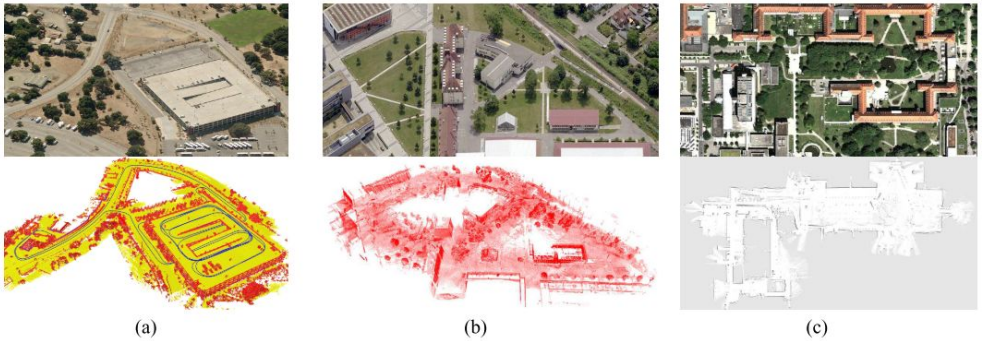
$$p(x_{1:T}, m \mid z_{1:T}, u_{1:T}, x_0).$$

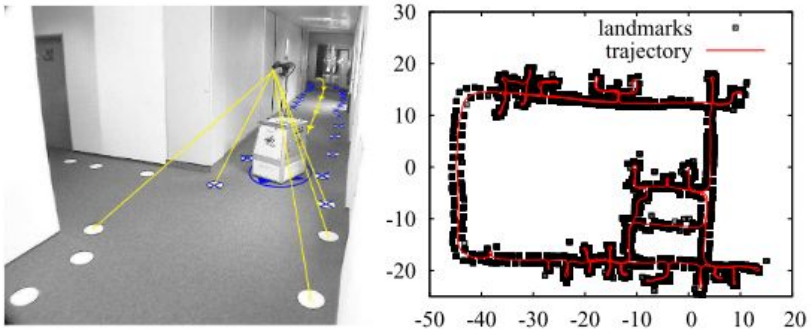
- Trajectory: $x_{1:T} = \{x_1, \ldots, x_T\}$
- Odometry measurements: $u_{1:T} = \{u_1, \ldots, u_T\}$
- Perceptions: $z_{1:T} = \{z_1, \ldots, z_T\}$.

Pose and Map Representation

The **poses** x1:T and the **odometry** u1:T are usually represented as 2D or 3D transformations in SE(2) or in SE(3), while the map can be represented in different ways.

Independently of the type of the representation, the map is defined by the **measurements** and the **locations** where these measurements have been acquired





Dynamic Bayesian network (DBN)

Transition model p(xt | xt−1, ut)

Observation model p(zt | xt,mt)

Expressing SLAM as a DBN highlights its temporal structure, and therefore this formalism is well suited to describe filtering processes that can be used to tackle the SLAM problem.

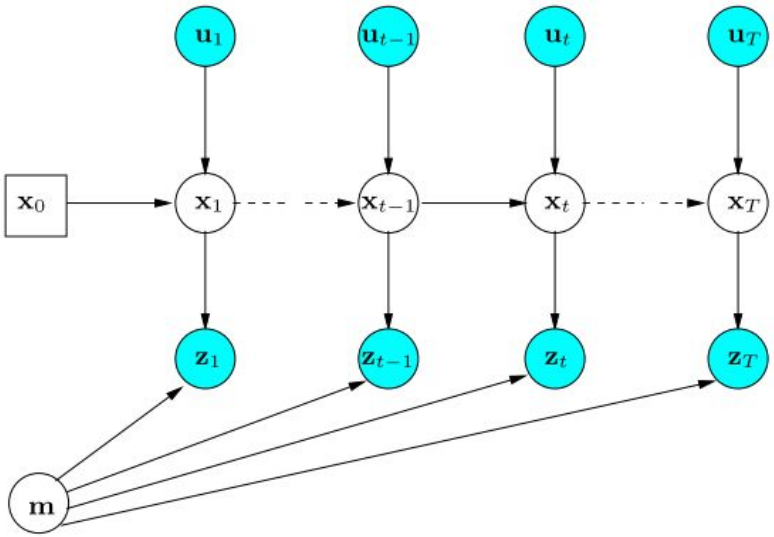


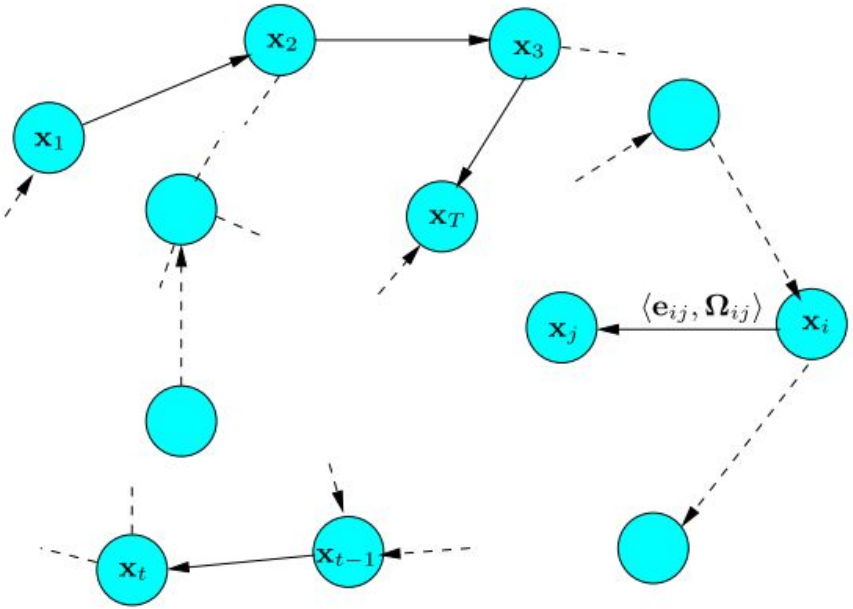
Fig. 4. Dynamic Bayesian Network of the SLAM process.

Graph-based SLAM

An alternative representation to the DBN is via the so-called “**graph-based**” or “**network-based**” formulation.

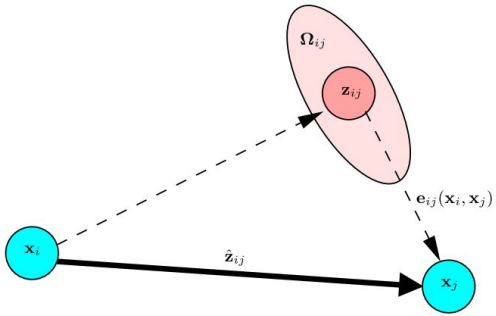
The **poses** of the robot are modeled by **nodes** in a graph and **labeled** with their position in the environment.

Spatial constraints between poses that result from **observations** zt or from **odometry measurements** ut are encoded in the **edges** between the nodes.



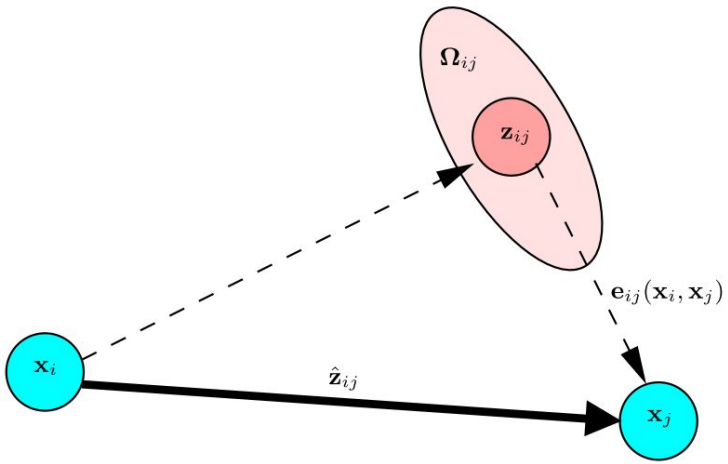
Graph-based SLAM

- graph-based SLAM the problem is decoupled in two tasks:
  - (Front-end) constructing the graph from the raw measurements (graph construction),
  - (Back-end) determining the most likely configuration of the poses given the edges of the graph (graph optimization).
- These raw measurements are replaced by the edges in the graph which can then be seen as "**virtual measurements**". An edge between two nodes is labeled with a probability distribution over the relative locations of the **two poses**, **conditioned to their mutual measurements**.



GRAPH-BASED SLAM

- x = (x1, . . . , xT)^T a vector of parameters
  - **xi** describes the pose of node i
  - **zij** the mean of a virtual measurement between node i and j
  - **Ωij** the information matrix of a virtual measurement
  - **zˆij(xi, xj)** prediction of a virtual measurement
- **virtual measurement** is a **transformation** that makes the observations acquired from i maximally **overlap** with the observation acquired from j. The log-likelihood lij of a measurement zij is therefore



$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$$

$$\mathrm{l}_{ij} \propto [\mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)]^T \mathbf{\Omega}_{ij} [\mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)]$$

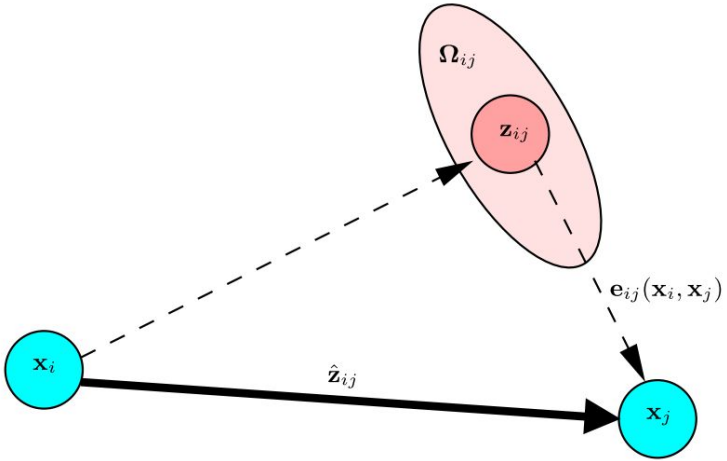
GRAPH-BASED SLAM

Let C be the set of pairs of indices for which a constraint (observation) z exists.

The goal of a maximum likelihood approach is to find the configuration of the nodes x^∗ that minimizes the negative log likelihood F(x) of all the observations

$$\mathbf{F}(\mathbf{x}) = \sum_{\langle i,j \rangle \in \mathcal{C}} \underbrace{\mathbf{e}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}}$$

$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}}\, \mathbf{F}(\mathbf{x}).$$



GRAPH-BASED SLAM

$$\mathbf{F}(\mathbf{x}) = \sum_{\langle i,j \rangle \in \mathcal{C}} \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}}$$

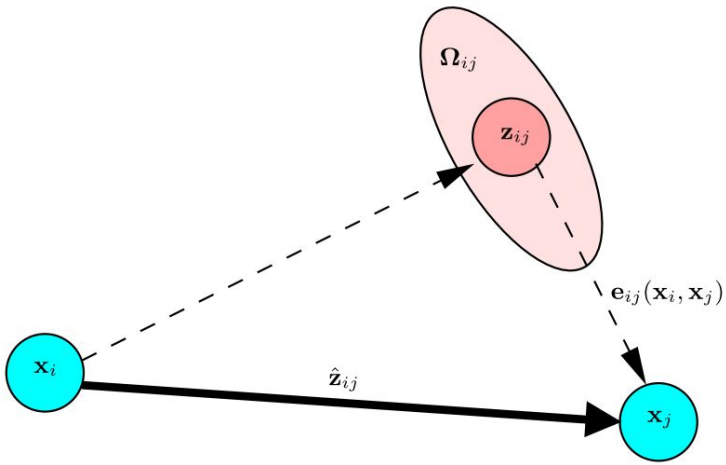
$$\mathbf{x}^* = \underset{\mathbf{x}}{\operatorname{argmin}}\, \mathbf{F}(\mathbf{x}).$$

$$\mathbf{e}_{ij}(\mathbf{x}_i, \mathbf{x}_j) = \mathbf{z}_{ij} - \hat{\mathbf{z}}_{ij}(\mathbf{x}_i, \mathbf{x}_j)$$

Can be obtained by using the Gauss-Newton or Levenberg-Marquardt algorithms.

The idea is to approximate the error function by its first order Taylor expansion around the current initial guess $\breve{\mathbf{x}}$

$$\begin{aligned} \mathbf{e}_{ij}(\breve{\mathbf{x}}_i + \boldsymbol{\Delta}\mathbf{x}_i, \breve{\mathbf{x}}_j + \boldsymbol{\Delta}\mathbf{x}_j) &= \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) \\ &\simeq \mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x}. \end{aligned}$$



GRAPH-BASED SLAM

$$
\begin{aligned}
&\mathbf{F}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) \\
&= \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x})^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) && (8)\\
&\simeq (\mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x})^T \boldsymbol{\Omega}_{ij} (\mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x}) && (9)\\
&= \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{c}_{ij}} + 2 \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}}_{\mathbf{b}_{ij}} \boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^T \underbrace{\mathbf{J}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{J}_{ij}}_{\mathbf{H}_{ij}} \boldsymbol{\Delta}\mathbf{x} && (10)\\
&= \mathbf{c}_{ij} + 2\mathbf{b}_{ij}\boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^T \mathbf{H}_{ij} \boldsymbol{\Delta}\mathbf{x} && (11)
\end{aligned}
$$

$$
\begin{aligned}
\mathbf{e}_{ij}(\breve{\mathbf{x}}_i + \boldsymbol{\Delta}\mathbf{x}_i, \breve{\mathbf{x}}_j + \boldsymbol{\Delta}\mathbf{x}_j) &= \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) \\
&\simeq \mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x}.
\end{aligned}
$$

$$
\mathbf{F}(\mathbf{x}) = \sum_{\langle i,j \rangle \in \mathcal{C}} \underbrace{\mathbf{e}_{ij}^T \boldsymbol{\Omega}_{ij} \mathbf{e}_{ij}}_{\mathbf{F}_{ij}}.
$$

$$
\begin{aligned}
\mathbf{F}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) &= \sum_{\langle i,j \rangle \in \mathcal{C}} \mathbf{F}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) && (12)\\
&\simeq \sum_{\langle i,j \rangle \in \mathcal{C}} \mathbf{c}_{ij} + 2\mathbf{b}_{ij}\boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^T \mathbf{H}_{ij} \boldsymbol{\Delta}\mathbf{x} && (13)\\
&= \mathbf{c} + 2\mathbf{b}^T \boldsymbol{\Delta}\mathbf{x} + \boldsymbol{\Delta}\mathbf{x}^T \mathbf{H} \boldsymbol{\Delta}\mathbf{x}. && (14)
\end{aligned}
$$

$$
\mathbf{c} = \sum \mathbf{c}_{ij}, \ \mathbf{b} = \sum \mathbf{b}_{ij}, \text{ and } \mathbf{H} = \sum \mathbf{H}_{ij}
$$

$$
\mathbf{H}\,\boldsymbol{\Delta}\mathbf{x}^* = -\mathbf{b}.
$$

GRAPH-BASED SLAM

$$\mathbf{H}\,\Delta\mathbf{x}^* = -\mathbf{b}. \qquad (15)$$

The linearized solution is then obtained by adding to the initial guess the computed increments x∗

$$\mathbf{x}^* = \breve{\mathbf{x}} + \Delta\mathbf{x}^*. \qquad (16)$$

Structure of the Linearized System

$$\mathbf{J}_{ij} = \begin{pmatrix} \mathbf{0} \cdots \mathbf{0} & \underbrace{\mathbf{A}_{ij}}_{\text{node } i} & \mathbf{0} \cdots \mathbf{0} & \underbrace{\mathbf{B}_{ij}}_{\text{node } j} & \mathbf{0} \cdots \mathbf{0} \end{pmatrix}. \quad (17)$$

$$\mathbf{H}_{ij} = \begin{pmatrix} \ddots & & & & \\ & \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{A}_{ij} & \cdots & \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{B}_{ij} & \\ & \vdots & \ddots & \vdots & \\ & \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{A}_{ij} & \cdots & \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{B}_{ij} & \\ & & & & \ddots \end{pmatrix} \quad (18)$$

$$\mathbf{b}_{ij} = \begin{pmatrix} \vdots \\ \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij} \\ \vdots \\ \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij} \\ \vdots \end{pmatrix} \quad (19)$$

Algorithm 1 Computes the mean $\mathbf{x}^*$ and the information matrix $\mathbf{H}*$ of the multivariate Gaussian approximation of the robot pose posterior from a graph of constraints.

Require: $\breve{\mathbf{x}} = \breve{\mathbf{x}}_{1:T}$: initial guess. $\mathcal{C} = \{\langle \mathbf{e}_{ij}(\cdot), \mathbf{\Omega}_{ij} \rangle\}$: constraints

Ensure: $\mathbf{x}^*$: new solution, $\mathbf{H}^*$ new information matrix

```
// find the maximum likelihood solution
while ¬converged do
    b ← 0        H ← 0
    for all ⟨e_ij, Ω_ij⟩ ∈ C do
        // Compute the Jacobians A_ij and B_ij of the error function
        A_ij ← ∂e_ij(x)/∂x_i |_{x=x̆}        B_ij ← ∂e_ij(x)/∂x_j |_{x=x̆}
        // compute the contribution of this constraint to the linear system
        H_[ii] += A_ij^T Ω_ij A_ij        H_[ij] += A_ij^T Ω_ij B_ij
        H_[ji] += B_ij^T Ω_ij A_ij        H_[jj] += B_ij^T Ω_ij B_ij
        // compute the coefficient vector
        b_[i] += A_ij^T Ω_ij e_ij        b_[j] += B_ij^T Ω_ij e_ij
    end for
    // keep the first node fixed
    H_[11] += I
    // solve the linear system using sparse Cholesky factorization
    Δx ← solve(H Δx = −b)
    // update the parameters
    x̆ += Δx
end while
x* ← x̆
H* ← H
// release the first node
H*_[11] −= I
return ⟨x*, H*⟩
```

Least Squares on a Manifold

An alternative idea is to consider the underlying space as a manifold and to define an operator $\boxplus$ that maps a local variation $\boldsymbol{\Delta}\mathbf{x}$ in the Euclidean space to a variation on the manifold, $\boldsymbol{\Delta}\mathbf{x} \mapsto \mathbf{x} \boxplus \boldsymbol{\Delta}\mathbf{x}$. We refer the reader to the work of Hertzberg [14] for the mathematical details. With this operator, a new error function can be defined as

$$\begin{aligned} \breve{\mathbf{e}}_{ij}(\boldsymbol{\Delta}\tilde{\mathbf{x}}_i, \boldsymbol{\Delta}\tilde{\mathbf{x}}_j) &\overset{\text{def.}}{=} \mathbf{e}_{ij}(\breve{\mathbf{x}}_i \boxplus \boldsymbol{\Delta}\tilde{\mathbf{x}}_i, \breve{\mathbf{x}}_j \boxplus \boldsymbol{\Delta}\tilde{\mathbf{x}}_j) && (20) \\ &= \mathbf{e}_{ij}(\breve{\mathbf{x}} \boxplus \boldsymbol{\Delta}\tilde{\mathbf{x}}) \simeq \breve{\mathbf{e}}_{ij} + \tilde{\mathbf{J}}_{ij}\boldsymbol{\Delta}\tilde{\mathbf{x}}, && (21) \end{aligned}$$

$$\begin{aligned} \mathbf{e}_{ij}(\breve{\mathbf{x}}_i + \boldsymbol{\Delta}\mathbf{x}_i, \breve{\mathbf{x}}_j + \boldsymbol{\Delta}\mathbf{x}_j) &= \mathbf{e}_{ij}(\breve{\mathbf{x}} + \boldsymbol{\Delta}\mathbf{x}) \\ &\simeq \mathbf{e}_{ij} + \mathbf{J}_{ij}\boldsymbol{\Delta}\mathbf{x}. \end{aligned}$$

where $\breve{\mathbf{x}}$ spans over the original over-parametrized space, for instance quaternions. The term $\boldsymbol{\Delta}\tilde{\mathbf{x}}$ is a small increment around the original position $\breve{\mathbf{x}}$ and is expressed in a minimal representation.

Algorithm 2 Manifold version of Algorithm 1. While this algorithm has the same computational complexity, it is substantially more robust than the non-manifold version, especially in the 3D case.

Require: $\breve{\mathbf{x}} = \breve{\mathbf{x}}_{1:T}$: initial guess. $\mathcal{C} = \{\langle \mathbf{e}_{ij}(\cdot), \mathbf{\Omega}_{ij} \rangle\}$: constraints

Ensure: $\mathbf{x}^*$: new solution, $\breve{\mathbf{H}}^*$ new information matrix

// find the maximum likelihood solution

while $\neg$converged **do**

// Compute the auxiliary Jacobians $M_{1:T}$ over the manifold

for all $\breve{\mathbf{x}}_i \in \breve{\mathbf{x}}$ **do**

$\mathbf{M}_i \leftarrow \left.\frac{\breve{\mathbf{x}}_i \boxplus \mathbf{\Delta}\tilde{\mathbf{x}}_i}{\partial \mathbf{\Delta}\tilde{\mathbf{x}}_i}\right|_{\mathbf{\Delta}\tilde{\mathbf{x}}=\mathbf{0}}$

end for

$\tilde{\mathbf{b}} \leftarrow \mathbf{0} \qquad \tilde{\mathbf{H}} \leftarrow \mathbf{0}$

for all $\langle \mathbf{e}_{ij}, \mathbf{\Omega}_{ij} \rangle \in \mathcal{C}$ **do**

// Compute the Jacobians $\mathbf{A}_{ij}$ and $\mathbf{B}_{ij}$ of the error function

$\mathbf{A}_{ij} \leftarrow \left.\frac{\partial \mathbf{e}_{ij}(\mathbf{x})}{\partial \mathbf{x}_i}\right|_{\mathbf{x}=\breve{\mathbf{x}}} \qquad \mathbf{B}_{ij} \leftarrow \left.\frac{\partial \mathbf{e}_{ij}(\mathbf{x})}{\partial \mathbf{x}_j}\right|_{\mathbf{x}=\breve{\mathbf{x}}}$

// Project the Jacobians through the manifold

$\tilde{\mathbf{A}}_{ij} \leftarrow \mathbf{A}_{ij}\mathbf{M}_i \qquad \tilde{\mathbf{B}}_{ij} \leftarrow \mathbf{B}_{ij}\mathbf{M}_j$

// compute the nonzero Hessian blocks

$\tilde{\mathbf{H}}_{[ii]} \mathrel{+}= \tilde{\mathbf{A}}_{ij}^T \mathbf{\Omega}_{ij} \tilde{\mathbf{A}}_{ij} \qquad \tilde{\mathbf{H}}_{[ij]} \mathrel{+}= \tilde{\mathbf{A}}_{ij}^T \mathbf{\Omega}_{ij} \tilde{\mathbf{B}}_{ij}$

$\tilde{\mathbf{H}}_{[ji]} \mathrel{+}= \tilde{\mathbf{B}}_{ij}^T \mathbf{\Omega}_{ij} \tilde{\mathbf{A}}_{ij} \qquad \tilde{\mathbf{H}}_{[jj]} \mathrel{+}= \tilde{\mathbf{B}}_{ij}^T \mathbf{\Omega}_{ij} \tilde{\mathbf{B}}_{ij}$

// compute the coefficient vector

$\tilde{\mathbf{b}}_{[i]} \mathrel{+}= \tilde{\mathbf{A}}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij} \qquad \tilde{\mathbf{b}}_{[j]} \mathrel{+}= \tilde{\mathbf{B}}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{e}_{ij}$

end for

// keep the first node fixed

$\mathbf{H}_{[11]} \mathrel{+}= \mathbf{I}$

// solve the linear system using sparse Cholesky factorization

$\mathbf{\Delta}\tilde{\mathbf{x}} \leftarrow \mathrm{solve}(\tilde{\mathbf{H}}\,\mathbf{\Delta}\tilde{\mathbf{x}} = -\tilde{\mathbf{b}})$

// update the parameters

for all $\breve{\mathbf{x}}_i \in \breve{\mathbf{x}}$ **do**

$\breve{\mathbf{x}}_i \leftarrow \breve{\mathbf{x}}_i \boxplus \mathbf{\Delta}\tilde{\mathbf{x}}_\mathbf{i}$

end for

end while

$\mathbf{x}^* \leftarrow \breve{\mathbf{x}}$

// the maximum is found, now compute the Hessian in the original space

$\mathbf{H}^* \leftarrow \mathbf{0}$

for all $\langle \mathbf{e}_{ij}, \mathbf{\Omega}_{ij} \rangle \in \mathcal{C}$ **do**

$\mathbf{H}_{[ii]} \mathrel{+}= \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{A}_{ij} \qquad \mathbf{H}_{[ij]} \mathrel{+}= \mathbf{A}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{B}_{ij}$

$\mathbf{H}_{[ji]} \mathrel{+}= \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{A}_{ij} \qquad \mathbf{H}_{[jj]} \mathrel{+}= \mathbf{B}_{ij}^T \mathbf{\Omega}_{ij} \mathbf{B}_{ij}$

end for

return $\langle \mathbf{x}^*, \mathbf{H}^* \rangle$

g2o: A general Framework for Graph Optimization

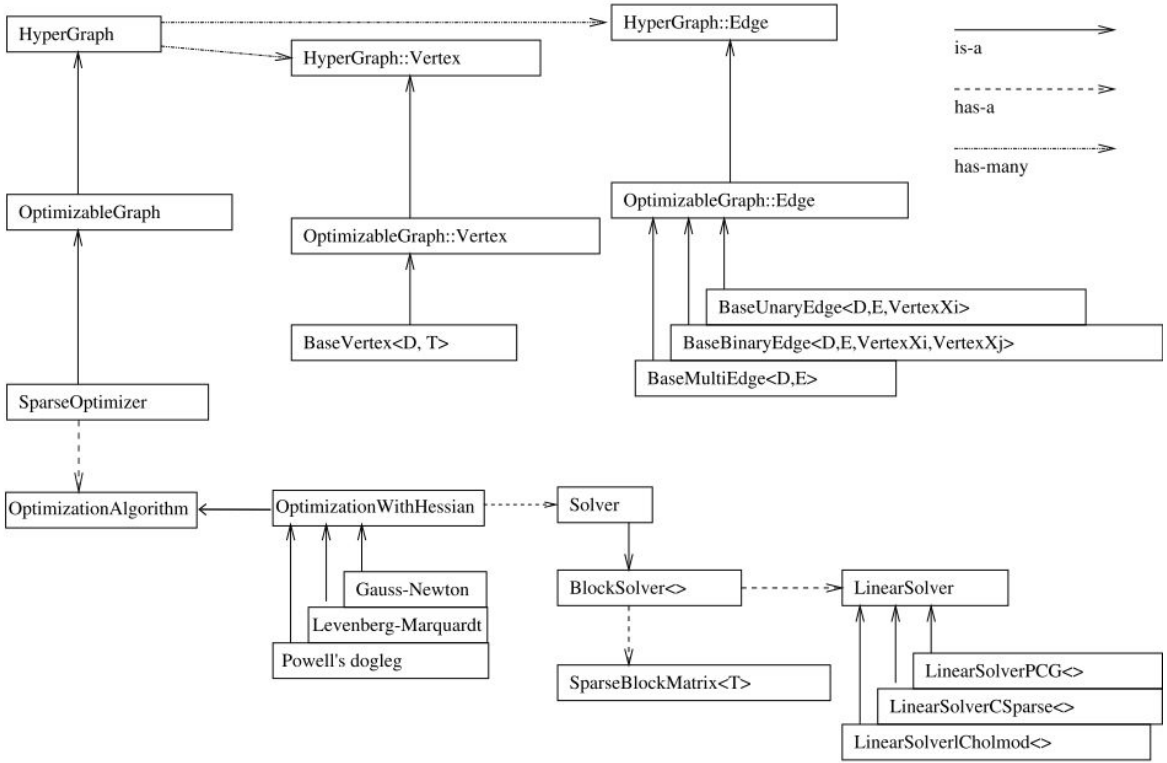


Figure 2: Class diagram of g^2o.

Design of the Error Functions

Variable	Symbol	Domain	Dimension	Parameterization of $\mathbf{\Delta x}$	$\boxplus$ operator
Robot pose	$\mathbf{x}_t^{\mathrm{s}}$	$SE(2)$	3	$(\Delta x\ \Delta y\ \Delta\theta)$	$\mathbf{x}_t^{\mathrm{s}} \oplus \mathbf{\Delta x}_t^{\mathrm{s}}$
Landmark pose	$\mathbf{x}_i^{\mathrm{l}}$	$\Re^2$	2	$(\Delta x\ \Delta y)$	$\mathbf{x}_i^{\mathrm{l}} + \mathbf{\Delta x}_i^{\mathrm{l}}$

Measurement	Symbol	Domain	Dimension	Set $\mathbf{x}_k$ of variables involved	error function
Odometry	$= \mathbf{z}_{t,t+1}^{\mathrm{s}}$	$SE(2)$	3	$\{\mathbf{x}_t^{\mathrm{s}}, \mathbf{x}_{t+1}^{\mathrm{s}}\}$	Eq. 42
Landmark	$= \mathbf{z}_{t,i}^{\mathrm{l}}$	$\Re^2$	2	$\{\mathbf{x}_t^{\mathrm{s}}, \mathbf{x}_i^{\mathrm{l}}\}$	Eq. 40

$$\mathbf{x}_t^{\mathrm{s}} \boxplus \mathbf{\Delta}\tilde{\mathbf{x}}_t^{\mathrm{s}} \doteq \begin{pmatrix} x + \Delta x \cos\theta - \Delta y \sin\theta \\ y + \Delta x \sin\theta + \Delta y \cos\theta \\ \mathrm{normAngle}(\theta + \Delta\theta) \end{pmatrix} \tag{36}$$

$$= \mathbf{x}_t^{\mathrm{s}} \oplus \mathbf{\Delta}\tilde{\mathbf{x}}_t^{\mathrm{s}}. \tag{37}$$

$$\mathbf{e}_{t,i}^{\mathrm{l}}(\mathbf{x}_t^{\mathrm{s}}, \mathbf{x}_i^{\mathrm{l}}) \doteq \mathbf{z}_{t,i} - \mathbf{h}_{t,i}^{\mathrm{l}}(\mathbf{x}_t^{\mathrm{s}}, \mathbf{x}_i^{\mathrm{l}}). \tag{40}$$

$$\mathbf{e}_{t,t+1}^{\mathrm{s}}(\mathbf{x}_t^{\mathrm{s}}, \mathbf{x}_{t+1}^{\mathrm{s}}) \doteq \mathbf{z}_{t,t+1} \ominus \mathbf{h}_{t,t+1}^{\mathrm{s}}(\mathbf{x}_t^{\mathrm{s}}, \mathbf{x}_{t+1}^{\mathrm{s}}). \tag{42}$$

Experiment

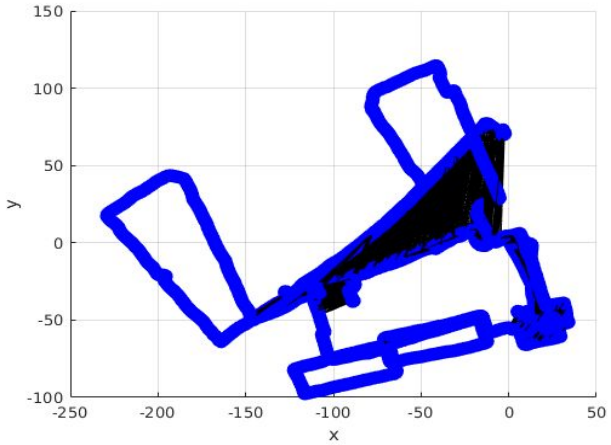
>> pg.graph

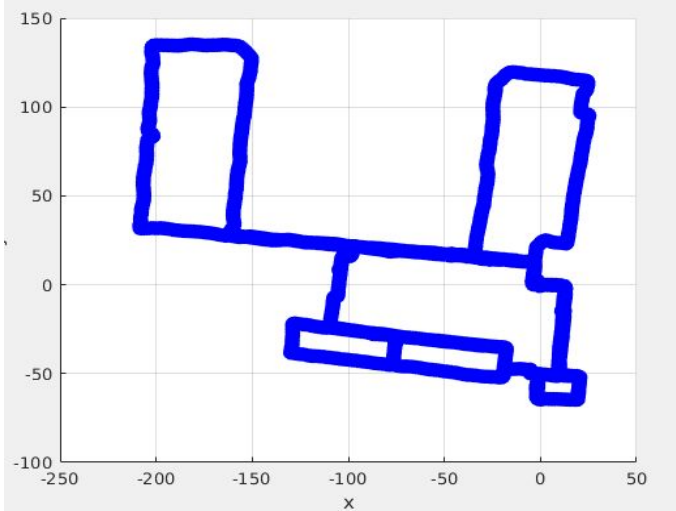
ans =

3 dimensions

1941 vertices

3995 edges





Experiment

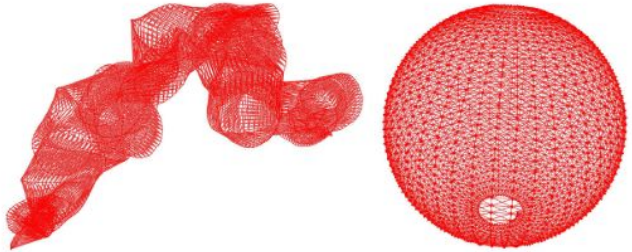


Fig. 11. Pose-graph obtained by simulating a robot moving on a sphere. Left: Initial configuration. Right: After optimizing the pose graph the sphere has accurately been recovered by Algorithm 2.

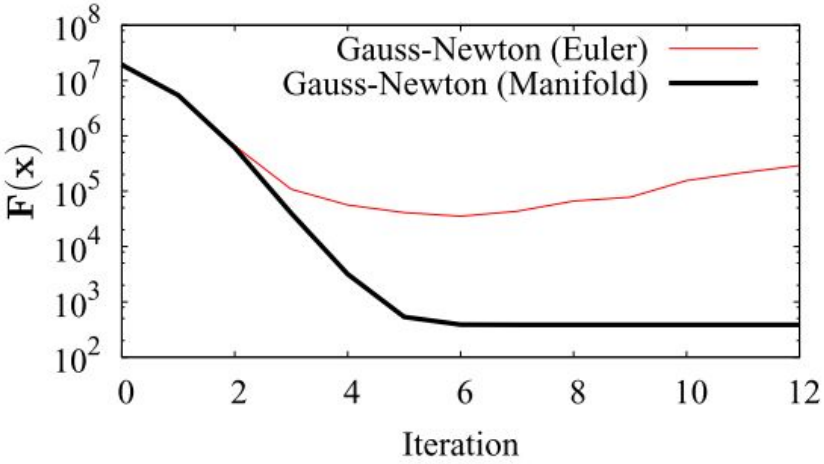


Fig. 12. Evolution of the error $\mathbf{F}(\mathbf{x})$ for Gauss-Newton optimization with Euler angles and with manifold linearization to the 3D sphere dataset.

Figure 12 shows the evolution of the error during the iterations of the two approaches. First both approaches are able to decrease the error. However, not appropriately considering the **singularities** leads to a divergence of Algorithm 1 while Algorithm 2 converges to the right solution.